CAMPUSCONNECT

Implementation Document

Submission owner: Rupam

Contact: drupam1289@gmail.com

Updated: 11 November 2026

Hosted application

https://campusconnect-rupam-2026.lucentterrapin1.chatgpt.site

Source repository

https://github.com/rupam1123/CampusConnect

This document describes the updated ChatGPT-hosted implementation. It supersedes the earlier MongoDB/SMTP deployment instructions in the supplied project documents.

1. Repository implementation

Frontend

frontend/src/App.jsx selects hosted authentication screens during the root production build. frontend/src/lib/api.js centralizes API configuration and response checking. Dashboard pages cover course rosters, projects, discussions, analytics and the forum.

Hosted backend

hosted/worker.mjs maps API methods and paths to shared controllers. It handles platform login context, profile creation, administrator permissions, record identifier validation and upload limits.

Shared behavior

backend/controllers implements course scheduling, enrollment, discussion replies, feedback aggregation, project applications and forum behavior. The original Express routes are retained for optional standalone hosting.

D1 adapter

hosted/store.mjs supports the query, population, save and update operations used by the controllers. Database access is scoped per request and database uniqueness is enforced by SQL indexes.

2. Build and local verification

Install dependencies with npm ci in the repository root, backend and frontend. Run npm run lint --prefix frontend, npm run build, then npm test.

Run npm start in the repository root to open the local QA preview on http://127.0.0.1:4173. Preview data is stored in outputs/preview.sqlite and excluded from source control.

The production build compiles the React assets and bundles a Worker ES module. The deployment archive contains the Worker, hosting configuration and SQL migrations. OWNER_EMAIL is set in hosted runtime secrets.

The automated flow suite uses Node test runner with an isolated in-memory SQLite implementation of the D1 interface. It performs real HTTP Request/Response calls against the bundled Worker.

3. Corrections made

Removed the IITK-only registration restriction for the hosted app. Added verified Gmail onboarding and server-side professor authorization. Fixed inconsistent or missing frontend API base URLs.

Added role, course membership and project ownership checks. Fixed account ID shape after profile updates, professor username collisions, schedule validation, feedback validation and course ownership on deletion.

Hosted feedback derives its course from the session instead of trusting the browser. Discussion replies are restricted to the same session. Session read records are deleted with the course.

The standalone authentication code rejects expired and cross-purpose verification codes. New professor records omit empty usernames, avoiding unique-index collisions. Account deletion uses unique anonymized identifiers and protects owned courses and projects.